ENHANCED SPLITWISE: AN INTELLIGENT EXPENSE MANAGEMENT AND GROUP SETTLEMENT PLATFORM

System Design, Implementation, and Evaluation of a Multi-Tier Intelligent Financial Platform

Chirag Bhalotia — 1032232301

Harsh Malu — 1032232445

Aditya Raj — 1032232452

Archit Kankaria — 1032231900

ABSTRACT

Personal and group expense management presents persistent friction across manual receipt logging, multi-participant shared bill division, transaction categorization, spending anomaly detection, and forward expenditure planning. Many existing commercial solutions demand tedious manual entry or attempt to employ generative artificial intelligence directly to compute currency splits, frequently resulting in arithmetic hallucinations and rounding discrepancies.

This report presents **Enhanced Splitwise (`enhanced-sw`)**, a comprehensive, multi-tiered financial management platform that integrates computer vision, large language model (LLM) routing, statistical analytics, and deterministic algorithms. The system decouples semantic perception from arithmetic execution. A local optical character recognition (OCR) engine processes receipt images with spatial reading-order sorting, while an LLM orchestration layer extracts structured line items into strict Pydantic schemas. All financial computations—including tax, tip, and discount allocations—are delegated to fixed-point Python `Decimal` algorithms with explicit rounding residual reconciliation down to the lowest currency unit ($\sum S_j = Total$).

Furthermore, the platform integrates automated zero-shot expense categorization across 11 canonical categories, qualitative spending insights synthesized from deterministic statistical metrics, monthly and category envelope budgeting, Leave-One-Out (LOO) statistical anomaly detection, time-series expenditure forecasting using Meta Prophet with 80% uncertainty intervals and adaptive cold-start fallbacks, and a Greedy Debt Simplification algorithm that resolves group obligations into a compact settlement plan with at most $N-1$ direct transfers. An interactive in-session settlement tracking workflow allows participants to record payment status and review remaining unsettled obligations. Automated testing validates platform stability across 109 passing Python test cases, 48 backend API integration tests, and 0 static TypeScript compilation errors.

Keywords: Expense Management, Optical Character Recognition, Large Language Models, Deterministic Arithmetic, Debt Simplification, Anomaly Detection, Meta Prophet Forecasting.

TABLE OF CONTENTS

Section	Title	Page
SECTION 1	**INTRODUCTION & PROBLEM FORMULATION**	**3**
1.1	Background & Motivation	3
1.2	Problem Statement	3
1.3	Core Project Objectives	3
1.4	Key Engineering Contributions	4
SECTION 2	**SYSTEM ARCHITECTURE & COMPONENT INTERACTION**	**4**
2.1	Multi-Tier Architectural Topology	4
2.2	Technology Stack & Design Rationale	5
2.3	Inter-Service Communication & Data Flow	5
SECTION 3	**RECEIPT OCR & DETERMINISTIC BILL SPLITTING**	**6**
3.1	Visual Capture & PaddleOCR Text Recognition	6
3.2	Spatial Reading-Order Layout Sorting	6
3.3	Resilient LLM Routing & Structured Extraction	6
3.4	Proportional Tax Allocation & Penny Reconciliation	7
SECTION 4	**CATEGORIZATION, BUDGETING & SPENDING INSIGHTS**	**8**
4.1	Automatic Classification into 11 Canonical Categories	8
4.2	Monthly & Category Envelope Budgeting	9
4.3	Decoupled Natural Language Spending Insights	10
SECTION 5	**STATISTICAL ANOMALIES & EXPENDITURE FORECASTING**	**10**
5.1	Leave-One-Out (LOO) Statistical Outlier Detection	10
5.2	Time-Series Expenditure Forecasting with Meta Prophet	11
SECTION 6	**GROUP DEBT SIMPLIFICATION & SETTLEMENT TRACKING**	**12**
6.1	Greedy Debt Simplification with Bipartite Matching	12
6.2	Interactive In-Session Settlement Tracking	13
SECTION 7	**DATA PERSISTENCE & API DESIGN**	**13**
7.1	Relational Schema & In-Memory Fallback Driver	13
7.2	Core API Endpoints & Contract Validation	14
7.3	Demo-Data Seeding Infrastructure	14
SECTION 8	**SYSTEM VERIFICATION & REPRESENTATIVE WORKFLOWS**	**15**
8.1	Multi-Tier Test Suite Execution	15
8.2	Algorithmic Computational Complexity Analysis	15
8.3	Representative User Interface Workflows	15
SECTION 9	**CRITICAL LIMITATIONS & FUTURE ROADMAP**	**21**
9.1	Architectural Scope & Inherent Limitations	22
9.2	Privacy, Data Protection & Security Profile	22
9.3	Future Engineering Roadmap	22
SECTION 10	**CONCLUSION & SELECT BIBLIOGRAPHY**	**22**
10.1	Project Summary	22
10.2	Select References	23

SECTION 1 — INTRODUCTION & PROBLEM FORMULATION

1.1 Background & Motivation

Effective personal financial management and transparent peer group accounting are essential requirements for modern individuals and households. However, managing daily expenditures across varied merchants and splitting complex group expenses (such as dining, shared utilities, grocery runs, and joint travel) remain fraught with friction. Users frequently encounter cognitive fatigue when deciphering physical or digital receipts, calculating individual shares under composite taxes, surcharges, and discounts, and tracking bilateral repayments across informal groups.

Traditional financial applications require exhaustive manual data entry or utilize rigid equal-split assumptions that fail when individuals order disparate items. More recently, emerging financial platforms have attempted to delegate calculations directly to Large Language Models (LLMs). While LLMs excel at unstructured semantic understanding, autoregressive neural networks inherently hallucinate numbers during multi-step carry arithmetic, leading to rounding errors and unacceptable balance discrepancies.

1.2 Problem Statement

A robust financial management platform must resolve three conflicting challenges:

1. **Unstructured Data Perception vs. Mathematical Exactness**: Ingestion must parse distorted, varied receipt layouts without human transcription, yet the resulting balances must obey strict conservation of currency where the sum of individual shares exactly equals the receipt total.
2. **Actionable Financial Guidance vs. Arithmetic Hallucination**: Users require contextual insights, category tracking, budget adherence alerts, and future spending projections without risking probabilistic arithmetic distortion.
3. **Multi-Party Debt Complexity vs. Usability**: Group debt obligations quickly degrade into dense, cyclic graphs of bilateral payments that confuse participants unless systematically simplified into a compact set of direct cash transfers.

1.3 Core Project Objectives

The `enhanced-sw` project was engineered to achieve the following tangible technical objectives:

- **Deterministic Bill Splitting**: Build an end-to-end receipt extraction and bill splitting pipeline that takes physical receipt images and natural language consumption instructions, extracts itemized records, and produces penny-balanced allocations using Python `Decimal` fixed-point arithmetic.
- **Intelligent Autonomous Categorization**: Categorize expenses automatically into 11 canonical accounting categories using zero-shot LLM reasoning with strict schema enforcement.
- **Explainable Anomaly Flagging**: Implement statistical outlier detection that isolates unusual category spending without allowing the outlier itself to skew the historical baseline.
- **Probabilistic Spend Projections**: Deploy Meta Prophet time-series models to forecast forward monthly expenditure trends with 80% uncertainty intervals while gracefully degrading when sample history is limited.
- **Greedy Debt Graph Settlement**: Reduce arbitrary *N*-person pairwise liabilities into a compact settlement plan with at most *N-1* direct cash transfers, accompanied by an interactive settlement tracking interface.

1.4 Key Engineering Contributions

Domain	Implemented Technical Mechanism	Primary Engineering Advantage
Receipt Extraction	PaddleOCR + Spatial Y-bucket sorting + Multi-provider LLM router	Resilient text recognition and schema parsing across orientation noise
Share Allocation	Pro-rata ratio mapping + Python `Decimal` rounding reconciliation	Deterministic rounding reconciliation ($\sum S_j \equiv Total, \Delta = 0.00$)
Outlier Detection	Leave-One-Out (LOO) statistical variance evaluation	Dynamic variance thresholding that eliminates baseline masking
Forecasting	Continuous daily resampling + Meta Prophet additive GAMs	Captures weekly seasonality and trend with 80% uncertainty intervals
Debt Simplification	Bilateral Max-Heap matching over net balance vectors	Reduces complex multigraph debts to at most N-1 direct cash transfers
Settlement Tracking	In-session client state tracking with remaining balance metrics	Provides real-time operational feedback on pending/settled transfers

SECTION 2 — SYSTEM ARCHITECTURE & COMPONENT INTERACTION

2.1 Multi-Tier Architectural Topology

The system is structured as a decoupled, multi-tiered enterprise architecture designed to maintain strict boundaries between client interaction, business orchestration, persistent storage, and analytical computation.

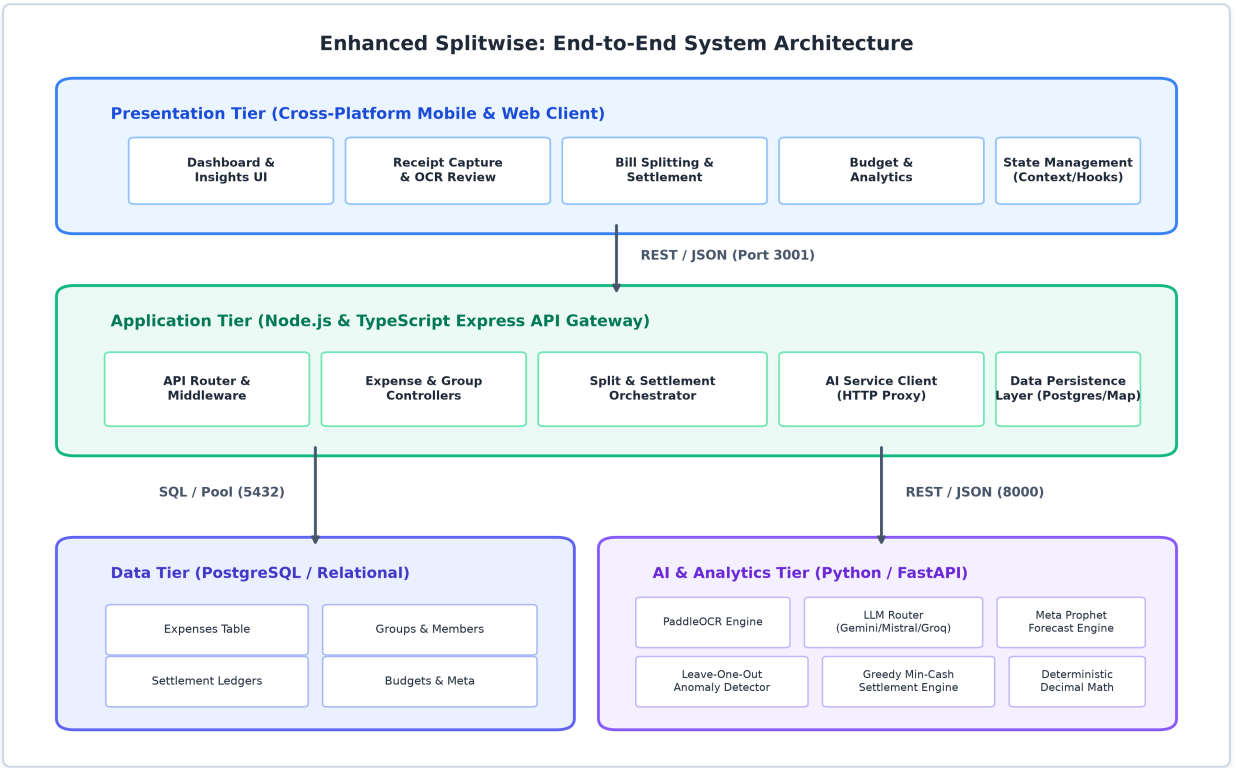


Figure 2.1: Architectural topology illustrating the four decoupled tiers of the enhanced-sw platform.

The platform partitions concerns across four distinct operational tiers:

1. **Presentation Tier (Client)**: Built with React Native and Expo (supporting Android, iOS, and Web environments). It provides an intuitive, Material-style responsive interface featuring custom bottom-bar navigation, modal sheets, and reactive state management.

2. **Application Gateway Tier (Orchestrator)**: Implemented in Node.js, Express, and TypeScript running on port 3001. It enforces REST endpoint contracts, performs request validation, manages database transactions, and securely proxies analytical queries to the AI microservice.
3. **Data Persistence Tier (Storage)**: A PostgreSQL 16 relational database maintaining tables for expenses and monthly budget configurations, backed by an automated in-memory repository driver for zero-dependency local operation and test execution.
4. **AI & Analytics Tier (Computation)**: A Python 3.12 and FastAPI microservice operating on port 8000. It hosts local PaddleOCR inference, multi-provider LLM routing via LangChain, Meta Prophet forecasting, and deterministic financial algorithms.

2.2 Technology Stack & Design Rationale

```
+-------------------------------------------------------------------------------+
|                              TECHNOLOGY STACK                                 |
+--------------------+----------------------------------------------------------+
| Presentation Tier  | React Native 0.76.7, Expo ~52.0.37, React 18.3.1, TypeScript |
| Gateway Tier       | Node.js 20+, Express 4.21, TypeScript 5.7, Multer 1.4     |
| Persistence Tier   | PostgreSQL 16 (pg driver), Automated SQL Migrations, Memory DB|
| AI & ML Analytics  | Python 3.12, FastAPI 0.115, PaddleOCR 2.10, Meta Prophet 1.4|
| LLM Orchestration  | LangChain Core, Google GenAI, MistralAI, Groq              |
| Test Frameworks    | Node.js Native Test Runner (tsx), Pytest 9.1, Puppeteer Core|
+--------------------+----------------------------------------------------------+
```

Design Rationale for Component Selection:

- **Decoupling Node.js and Python**: Node.js and TypeScript provide exceptional asynchronous I/O performance and strong static typing for user-facing REST APIs and data persistence. Conversely, Python remains the premier ecosystem for scientific computing, OCR models, and statistical time-series forecasting. Separating these environments allows each runtime to scale independently without memory bloat.
- **Local OCR vs. Cloud Document APIs**: PaddleOCR runs locally on standard CPU hardware without incurring per-image cloud billing, preserving user privacy and operational autonomy during receipt capture.
- **Stateless Computational AI Layer**: The Python FastAPI service maintains zero database state. It receives transaction lists or participant arrays via HTTP POST, computes mathematical or statistical outcomes deterministically, and returns validated JSON payloads. This supports horizontal scalability and straightforward unit testing.

2.3 Inter-Service Communication & Data Flow

All client-to-backend requests target the Application Gateway (`http://localhost:3001`). The Gateway validates request headers, enforces payload sanitization, interacts with PostgreSQL when persistence is required, and marshals analytical payloads to the Python service (`http://localhost:8000`).

```
[Client / Mobile App]
       |  (Multipart Form-Data / JSON REST)
       v
[Application Gateway :3001]
       |
       +---> [PostgreSQL Persistence :5432] (Expenses, Budgets, Migrations)
       |
       +---> [Python AI Microservice :8000]
                    |
                    +---> [PaddleOCR Engine] (Local CPU text detection & recognition)
                    +---> [Multi-Provider LLM Router] (Gemini / Mistral / Groq fallback)
                    +---> [Decimal Math Engine] (Penny-balanced tax/split allocations)
                    +---> [Prophet & LOO Analytics] (Forecasting & Outlier scoring)
                    +---> [Greedy Simplifier] (Min-heap bilateral cash matching)
```

SECTION 3 — RECEIPT OCR & DETERMINISTIC BILL SPLITTING

3.1 Visual Capture & PaddleOCR Text Recognition

When a user captures or uploads a bill image via the mobile interface, the client transmits the image as multipart form-data to the backend gateway, which forwards it to the Python service endpoint `/api/bill/process`.

Text extraction utilizes **PaddleOCR**, an efficient, lightweight optical character recognition architecture. The engine executes a two-stage computer vision pipeline:

1. **Text Detection (DBNet)**: Real-time differentiable binarization detects arbitrary text bounding boxes *[x_min, y_min, x_max, y_max]* regardless of aspect ratios or thermal receipt wrinkles.
2. **Text Recognition (SVTR)**: A single visual model with text-line pooling decodes alphanumeric text and price glyphs from detected bounding boxes, attaching individual detection confidence scores.

3.2 Spatial Reading-Order Layout Sorting

Raw OCR bounding boxes are emitted in visual detection order, which frequently scrambles line-item names and their corresponding prices on two-column cash register receipts. To reconstruct natural reading order prior to language model parsing, `enhanced-sw` applies a spatial bucket clustering algorithm:

```
Algorithm 1: Spatial Reading-Order Layout Sorter
Input: Tokens T = {(text_k, box_k, confidence_k)}
Output: Formatted multi-line text buffer

1: Sort tokens T primarily by vertical top coordinate box.y_top ascending
2: Initialize empty Lines array
3: For each token t in T:
4:     Find line L in Lines where |t.box.y_top - L.mean_y| <= 10.0 pixels
5:     If matching line L exists:
6:         Append token t to L and update L.mean_y
7:     Else:
8:         Create new line L_new = [t] with mean_y = t.box.y_top
9: Sort all Lines by mean_y ascending
10: For each Line L:
11:    Sort tokens in L horizontally by box.x_left ascending
12: Return joined lines separated by newline characters
```

This spatial sorting groups horizontally aligned receipt text (such as "Garlic Naan" and "200.00") onto the identical text line, drastically reducing semantic parsing ambiguity.

3.3 Resilient LLM Routing & Structured Extraction

Once reconstructed, the text buffer and the user's natural language assignment prompt (e.g., *"Arun and Sneha shared the starters, Vikram had the pasta"*) are dispatched to the `LLMRouter`.

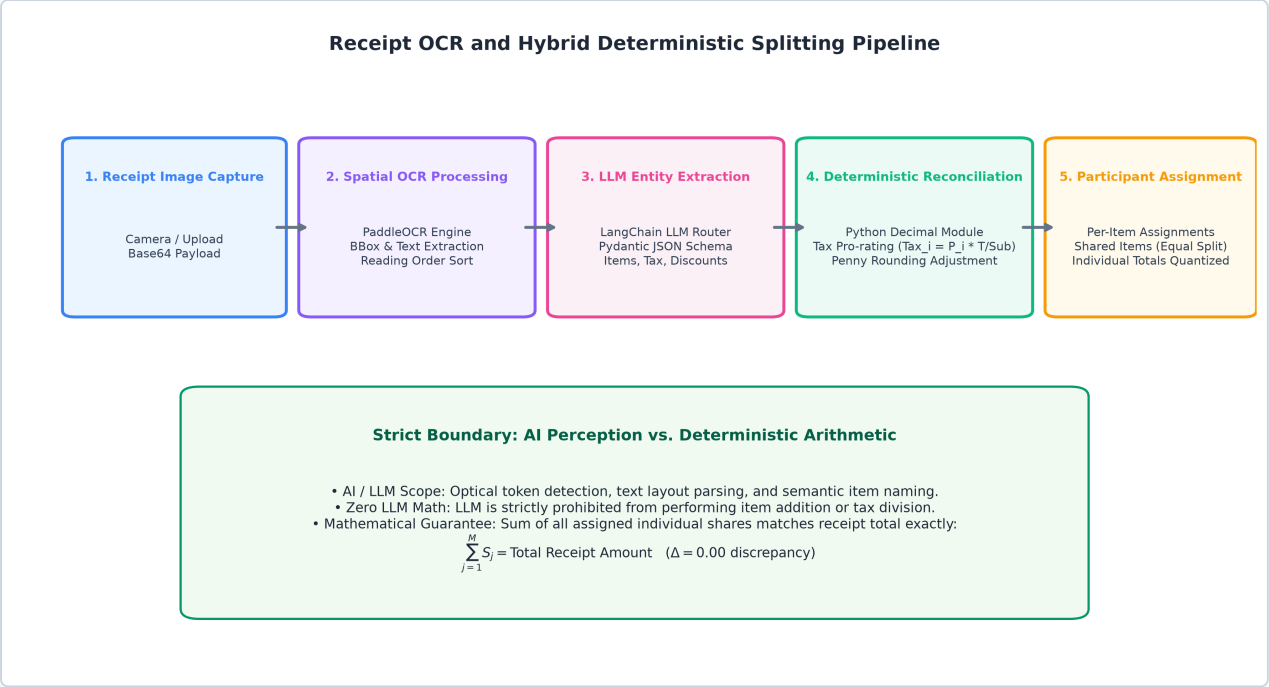


Figure 3.1: Hybrid bill splitting pipeline enforcing the boundary between LLM semantic perception and deterministic Decimal reconciliation.

Multi-Provider Fallback Architecture

To improve operational availability against transient API outages or rate limits, the `LLMRouter` implements an automated fallback chain across three independent language model providers:

1. **Primary**: Google Gemini (`gemini-3.6-flash` via `langchain_google_genai.ChatGoogleGenerativeAI`).
2. **Secondary**: Mistral AI (`mistral-small-latest` via `langchain_mistralai.ChatMistralAI`).
3. **Tertiary**: Groq Cloud (`openai/gpt-oss-20b` via `langchain_groq.ChatGroq`).

If the primary provider encounters HTTP 429 rate-limiting, quota exhaustion, or connectivity timeouts, the router catches the exception and attempts execution through the next configured provider. The resulting JSON payload is strictly coerced into a structured **Pydantic** schema:

```
class ParsedBillItem(BaseModel):
    name: str = Field(..., description="Name of the line item")
    price: float = Field(..., ge=0, description="Item base price")
    quantity: int = Field(default=1, ge=1)
    assigned_to: List[str] = Field(..., description="List of participant names")

class ParsedBill(BaseModel):
    merchant: Optional[str] = None
    subtotal: float
    tax: float = 0.0
    discount: float = 0.0
    tip: float = 0.0
    total: float
    items: List[ParsedBillItem]
```

3.4 Proportional Tax Allocation & Penny Reconciliation

Crucially, **the LLM is strictly prohibited from calculating final per-person monetary totals**. LLMs frequently make simple arithmetic errors when distributing taxes, service charges, and discounts across unevenly shared items. Instead, all parsed figures are converted into Python `Decimal` instances with fixed two-decimal-place quantization (`ROUND_HALF_UP`).

Mathematical Formulation of Share Allocation

Let I be the set of parsed line items, and let P denote the set of participants. For each item $i \in I$ with base cost C_i and assigned consumer subset $P_i \subseteq P$, the unweighted item share assigned to participant $j \in P_i$ is:

$$S_{i,j} = (C_i / |P_i|)$$

The pre-tax, pre-discount subtotal for participant j is given by:

$$Subtotal_j = \sum_{i \in I \,:\, j \in P_i} S_{i,j}$$

Receipt surcharges and credits (Tax T, Tip Tip, Discount D) yield a net multiplier ratio ρ relative to the receipt base subtotal $B = \sum_{i \in I} C_i$:

$$\rho = \frac{(T + Tip - D)}{B}$$

The preliminary allocated share for participant j is calculated as:

$$\widetilde{S}_j = round\Big(Subtotal_j \cdot (1 + \rho), \, 2\Big)$$

Exact Penny Rounding Reconciliation

Because independent rounding of rounded floating fractions can cause $\sum_{j \in P} \widetilde{S}_j \neq Total$, the deterministic engine calculates the exact residual discrepancy:

$$\Delta = Total - \sum_{j \in P} \widetilde{S}_j$$

If $\Delta \neq 0.00$, the engine reconciles the difference in increments of the lowest currency unit (0.01). If $\Delta > 0$, pennies are systematically allocated to the highest-paying participant shares until the discrepancy is eliminated; if $\Delta < 0$, pennies are similarly subtracted. This deterministic adjustment ensures that the quantized shares reconcile exactly to the evaluated receipt total ($\Delta \equiv 0.00$).

SECTION 4 — CATEGORIZATION, BUDGETING & SPENDING INSIGHTS

4.1 Automatic Classification into 11 Canonical Categories

Every recorded expense requires appropriate categorization for meaningful budgeting and analytics. When an expense is recorded without an explicit category, the system routes the description and merchant to the AI microservice (`/api/expense/categorize`).

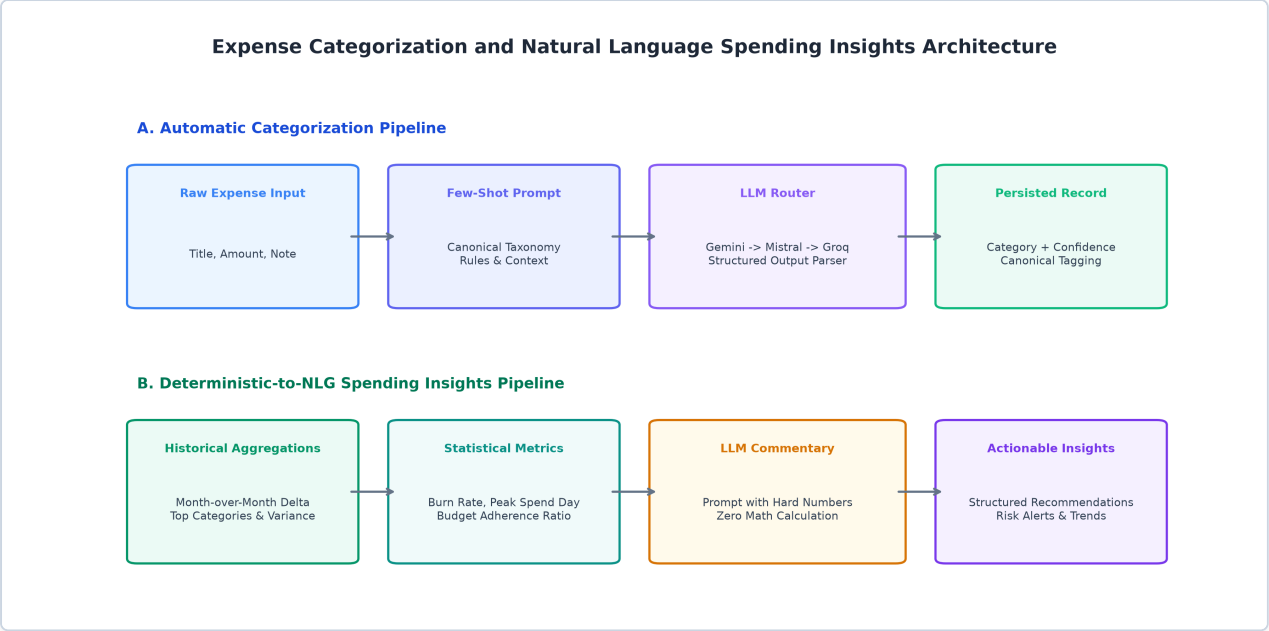


Figure 4.1: Dual pipeline for automated transaction categorization and decoupled qualitative insight synthesis.

The platform enforces a standardized taxonomy of **11 canonical expense categories**:

1. `Food & Dining`
2. `Transportation`
3. `Shopping`
4. `Entertainment`
5. `Bills & Utilities`
6. `Healthcare`
7. `Education`
8. `Travel`
9. `Groceries`
10. `Personal Care`
11. `Other`

The LLM is provided with contextual category definitions and constrained via Pydantic to select exactly one valid category along with an associated confidence score and a concise rationale. If the model fails or outputs an unrecognized string, the system safely defaults to `Other`.

4.2 Monthly & Category Envelope Budgeting

The persistence tier stores monthly spending targets in the `budgets` table keyed by month (`YYYY-MM`). Each record defines an overall monthly target alongside an optional JSON dictionary of category-specific spending caps.

The Node.js `BudgetService` computes deterministic budget progress on demand:

- **Total Consumption**: Sums all persisted expenses for the calendar month and calculates the remaining balance and percentage utilized.
- **Envelope Health Indicators**: Evaluates category-level expenditure against assigned caps, tagging each category with status indicators:
- `OK` (< *80%* consumed)
- `WARNING` (*80%* - *100%* consumed)
- `EXCEEDED` (> *100%* consumed)

4.3 Decoupled Natural Language Spending Insights

To generate narrative monthly financial summaries, the application adheres to a decoupled architecture:

1. **Deterministic Aggregation**: The backend queries all transactions for the specified month and computes hard mathematical metrics directly: total expenditure, top spending category, category percentage breakdown, daily burn rate, and projected end-of-month spend.
2. **Qualitative Synthesis**: These verified metrics are supplied as structured parameters into a tailored prompt for the LLM. The model generates qualitative narrative commentary, identifies spending trends, and offers practical savings advice without performing any independent arithmetic.

SECTION 5 — STATISTICAL ANOMALIES & EXPENDITURE FORECASTING

5.1 Leave-One-Out (LOO) Statistical Outlier Detection

Traditional anomaly detection often compares a candidate transaction x_t against the mean μ and standard deviation σ of the entire historical sample. However, an extreme outlier can artificially inflate both μ and σ, thereby masking its own anomalous nature.

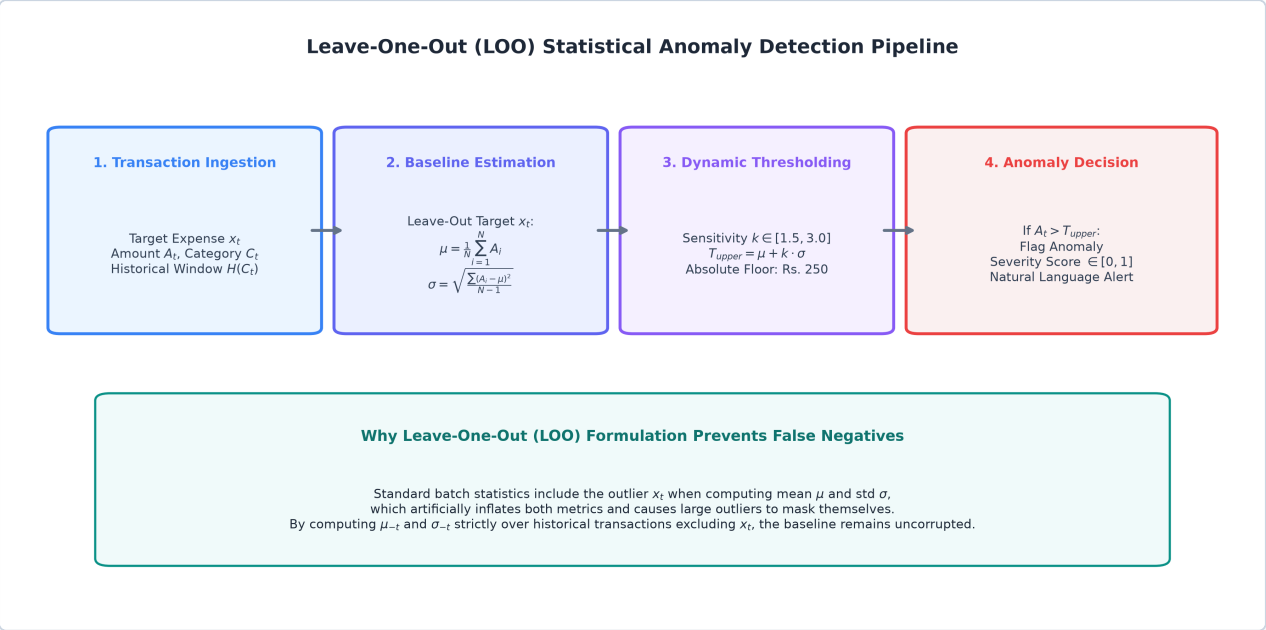


Figure 5.1: Leave-One-Out (LOO) statistical outlier detection pipeline preventing baseline contamination.

To solve this, `enhanced-sw` implements a **Leave-One-Out (LOO)** statistical evaluation across category histories:

```
Algorithm 2: Leave-One-Out Category Outlier Evaluation
Input: Category historical transactions H = {x_1, x_2, ..., x_N}, Candidate item x_t
Parameters: Sensitivity factor k = 2.5, Absolute floor T_floor = ₹250.00
Output: is_anomaly (bool), severity (str), reason (str)

1: If |H| < 3: Return (False, "NONE", "Insufficient sample size")
2: Compute LOO baseline excluding x_t: H_loo = H \ {x_t}
3: mu_loo = Mean(H_loo)
4: sigma_loo = StandardDeviation(H_loo)
5: Dynamic threshold T_upper = Max(mu_loo + (k * Max(sigma_loo, 0.3 * mu_loo)), T_floor)
6: If x_t > T_upper AND x_t >= T_floor:
7:     ratio = x_t / mu_loo
8:     If x_t >= mu_loo + 3.0 * sigma_loo OR ratio >= 3.0: severity = "HIGH"
9:     Else If x_t >= mu_loo + 2.0 * sigma_loo OR ratio >= 2.0: severity = "MEDIUM"
10:    Else: severity = "MILD"
11:    Return (True, severity, Format("₹{x_t} is {ratio:.1f}x higher than category baseline"))
12: Return (False, "NONE", "Transaction within expected variance")
```

Because this is a statistical heuristic rather than a heavy black-box machine learning model, anomaly flags are fully transparent, computationally instantaneous ($O(N)$), and accompanied by plain-English explanations.

5.2 Time-Series Expenditure Forecasting with Meta Prophet

Predicting forward expenses requires modeling underlying spending trajectories alongside periodic weekly spikes (e.g., weekend leisure or grocery runs).

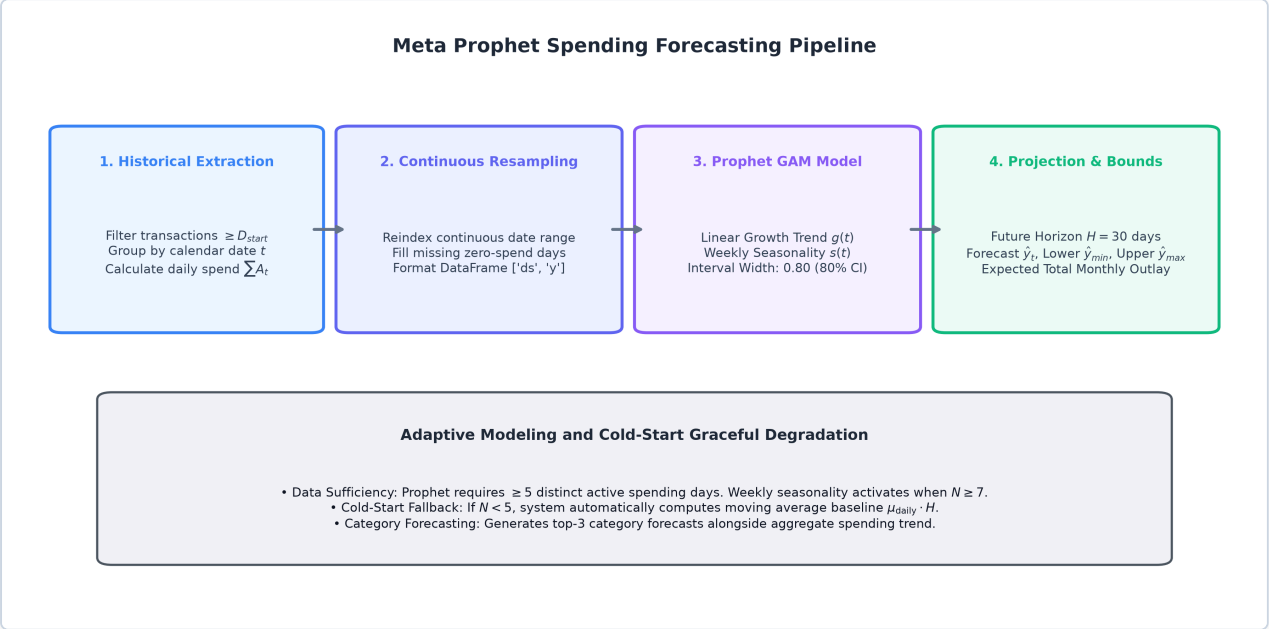


Figure 5.2: Continuous time-series preprocessing, Prophet GAM fitting, and cold-start fallback flow.

Preprocessing & Continuous Daily Resampling

Expense timestamps are aggregated by calendar day. Because consumers do not transact every single day, standard raw series contain temporal gaps. The service reindexes the date index continuously over the entire historical range, explicitly imputing zero spend (`0.00`) for non-active days. This prevents the model from misinterpreting missing dates as compressed time intervals.

Model Specification & Uncertainty Bounds

The forecasting engine deploys **Meta Prophet**, a Generalized Additive Model (GAM) defined as:

$$y(t) = g(t) + s(t) + \varepsilon_t$$

where *g(t)* represents a piecewise linear growth trend, *s(t)* models weekly Fourier seasonality (*P = 7* days), and *ε_t* captures residual variance.

- **Uncertainty Intervals**: Configured at an **80% uncertainty interval** (`interval_width=0.80`), producing upper and lower forward bounds (*ŷ_lower, ŷ_upper*).
- **Gating & Fallback Behavior**: Time-series fitting requires adequate temporal structure:
- **Data Sufficiency Threshold**: The service requires at least 5 valid expense transactions spanning at least 3 distinct calendar days. If these thresholds are not satisfied, the forecaster returns `has_sufficient_data = False` with an empty forecast points list (`[]`) and zero projected expenditure, explicitly signaling to the client that sample history is insufficient for projection.
- **Model Fitting Exception Fallback**: When sufficient data is present but Prophet model optimization fails (e.g., numerical convergence issues or linear algebra exceptions during MCMC/L-BFGS), the service executes a graceful fallback (`_build_fallback_response`). This baseline projects future spending using a deterministic 7-day historical daily average with heuristic *\pm 30%* uncertainty bounds (`lower = 0.7 pred`, `upper = 1.3 pred`), labeled as `Baseline (Prophet Fallback)`.

SECTION 6 — GROUP DEBT SIMPLIFICATION & SETTLEMENT TRACKING

6.1 Greedy Debt Simplification with Bipartite Matching

In a peer group where multiple members pay for varied activities, informal IOUs create dense, circular debt graphs. For *N* participants, arbitrary bilateral agreements can require up to *(N(N-1) / 2)* separate transactions.

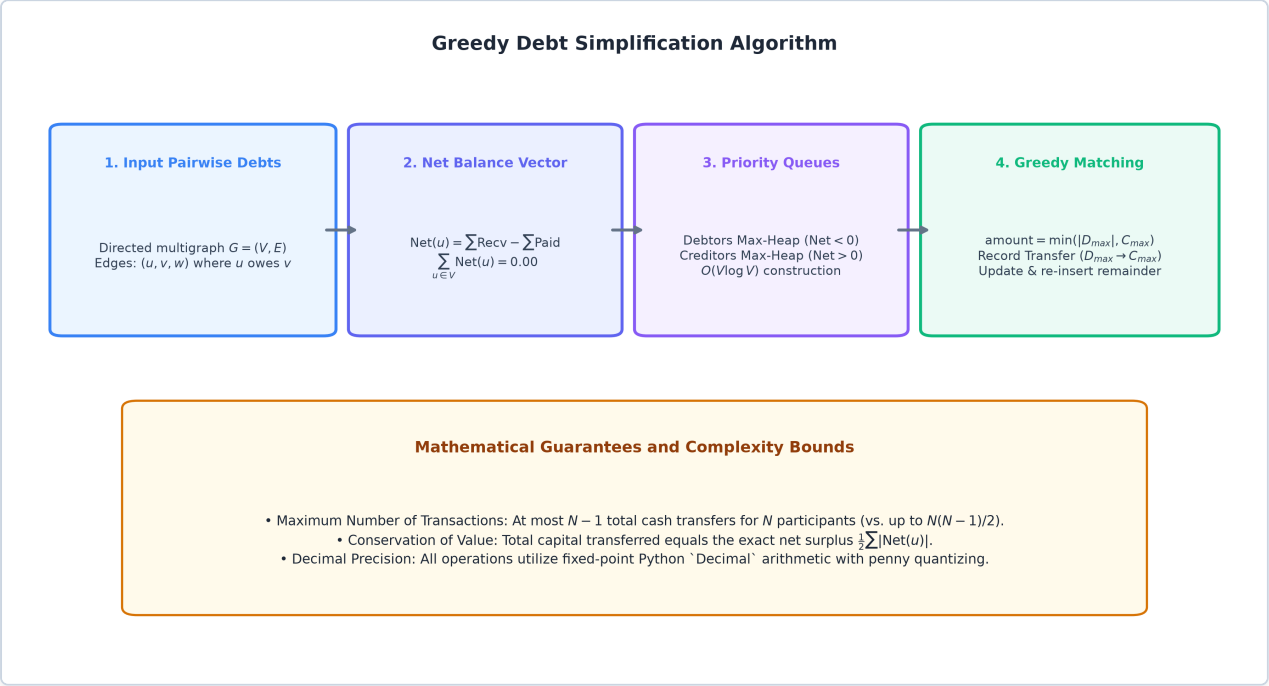


Figure 6.1: Greedy Debt Simplification Algorithm showing net balance extraction, max-heaps, and greedy settlement matching.

Algorithmic Execution

The debt simplifier models the group as a directed cash-flow network and reduces it to a compact settlement plan with at most *N-1* direct transfers:

1. **Net Balance Vector Extraction**: For each participant *u*, compute net position: *Net(u) = Amount Paid_u - Fair Share_u*. Conservation of currency ensures *∑_u ∈ V Net(u) = 0.00*. Participants with *Net < 0* are net debtors; those with *Net > 0* are net creditors.

2. **Priority Queue Matching**: Debtors and creditors are inserted into two distinct Max-Heaps ordered by absolute magnitude.
3. **Greedy Bilateral Elimination**: At each step, pop the largest debtor *D_max* and largest creditor *C_max*. Settle *transfer_amount = min(|D_max|, C_max)*, emit the transfer, and reinsert any remaining balance back into the corresponding heap.

```
Algorithm 3: Greedy Debt Simplification
Input: Participant payments List P = {(person_i, amount_paid_i)}
Output: Compact transfers List T = {(from_person, to_person, amount, instruction)}

1: total_spent = Sum(p.amount_paid for p in P)
2: fair_share = total_spent / |P|
3: Initialize MaxHeap Debtors, Creditors
4: For each p in P:
5:     net = p.amount_paid - fair_share
6:     If net < -0.01: Debtors.Push(|net|, p.person)
7:     Else If net > 0.01: Creditors.Push(net, p.person)
8: Initialize Transfers = []
9: While Debtors and Creditors are not empty:
10:    (d_amt, debtor) = Debtors.PopMax()
11:    (c_amt, creditor) = Creditors.PopMax()
12:    settle_amt = Min(d_amt, c_amt)
13:    Append (debtor, creditor, settle_amt) to Transfers
14:    If d_amt - settle_amt > 0.01: Debtors.Push(d_amt - settle_amt, debtor)
15:    If c_amt - settle_amt > 0.01: Creditors.Push(c_amt - settle_amt, creditor)
16: Return Transfers
```

6.2 Interactive In-Session Settlement Tracking

Generating transfers resolves mathematical debts, but participants need an intuitive workflow to track payments in progress.

In-Session Status Workflow

The mobile client provides interactive settlement tracking directly on calculated results:

- **Pending vs. Settled Status**: Each transfer card features an interactive checkmark allowing users to toggle status between `Pending` and `Settled`. Settled transfers are visually struck through and badged in green.
- **Dynamic Real-Time Overview**: The interface recalculates the count of pending and settled transfers, the remaining unsettled cash balance, and displays an all-settled completion banner when all transfers are marked settled.
- **Architectural Scope & Session Limitation**: The settlement engine is completely **stateless**. No persistent group ledger or multi-user identity system exists in the database. Consequently, settlement tracking is maintained **in client memory during the active session only** and is reset upon navigating away or restarting the application. The system does not verify actual UPI, bank, or payment transfers.

SECTION 7 — DATA PERSISTENCE & API DESIGN

7.1 Relational Schema & In-Memory Fallback Driver

The persistence architecture utilizes PostgreSQL with automated SQL schema migrations executed during gateway startup.

```
-- Core Relational Tables (PostgreSQL 16)
CREATE TABLE IF NOT EXISTS expenses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    description TEXT NOT NULL CHECK (length(trim(description)) > 0),
    amount NUMERIC(12, 2) NOT NULL CHECK (amount >= 0),
    category TEXT NOT NULL CHECK (category IN (
        'Food & Dining', 'Transportation', 'Shopping', 'Entertainment',
        'Bills & Utilities', 'Healthcare', 'Education', 'Travel',
        'Groceries', 'Personal Care', 'Other'
    )),
    merchant TEXT,
    expense_date DATE NOT NULL DEFAULT CURRENT_DATE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS budgets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    month_year VARCHAR(7) NOT NULL UNIQUE, -- 'YYYY-MM'
    monthly_budget NUMERIC(12, 2) NOT NULL CHECK (monthly_budget >= 0),
    category_budgets JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

In-Memory Database Fallback

To facilitate zero-configuration local development, CI testing, and seamless demonstration without requiring a live PostgreSQL instance, the database client (`client.ts`) includes an automatic fallback driver. If connecting to PostgreSQL fails, the repository seamlessly switches to an in-memory data store that emulates SQL operations and UUID generation.

7.2 Core API Endpoints & Contract Validation

Route	Method	Tier / Service	Responsibility
`/api/expenses`	`GET` , `POST`	Gateway → DB	List expenses; create expense with automatic categorization fallback
`/api/expenses/:id`	`DELETE`	Gateway → DB	Remove recorded expense item
`/api/budgets/:month`	`GET` , `POST`	Gateway → DB	Retrieve monthly budget status; upsert monthly & category targets
`/api/bill/process`	`POST`	Gateway → AI	Upload receipt image and prompt; returns item shares & tax breakdown
`/api/expense/categorize`	`POST`	Gateway → AI	Zero-shot LLM classification into 11 canonical categories
`/api/spending/insights`	`GET` , `POST`	Gateway → AI	Computes monthly burn rate, category health, and LLM advice
`/api/spending/forecast`	`POST`	Gateway → AI	Computes 30-day Meta Prophet spending forecast and bounds
`/api/settlement/simplify`	`POST`	Gateway → AI	Stateless greedy debt simplification across participant payments
`/api/dev/seed`	`POST`	Gateway → DB	Seeds realistic 60-day transactions and demo budget data

7.3 Demo-Data Seeding Infrastructure

For viva demonstrations and evaluation, the backend includes an automated seeder (`POST /api/dev/seed`) that populates the repository with over 45 realistic transactions spanning the past 60 days. The seeded dataset includes diverse merchants, authentic weekend spending spikes for Prophet time-series modeling, intentional statistical anomalies (e.g., a ₹9,800 banquet dinner and a ₹14,500 gaming keyboard), and an active monthly budget.

SECTION 8 — SYSTEM VERIFICATION & REPRESENTATIVE WORKFLOWS

8.1 Multi-Tier Test Suite Execution

Platform stability was verified across both backend and AI service runtimes using dedicated automated test suites.

```
======================== AUTOMATED TEST SUITE SUMMARY ========================
1. Python AI & Analytics Test Suite (pytest 9.1):
   - 109 Passed, 6 Skipped (live API keys), 0 Failed across 115 test cases
   - Verified: OCR layout sorting, Pydantic schemas, Decimal penny reconciliation,
     LOO anomaly scoring, Meta Prophet forecasting, and debt simplification.

2. Node.js Gateway & Persistence Test Suite (npm test):
   - 48 Passed, 0 Failed across 9 test suites
   - Verified: REST routing, input validation, in-memory repository CRUD,
     category normalization, budget calculations, and proxy error handling.

3. Frontend Static Type Checking (tsc --noEmit):
   - 0 TypeScript compilation errors across all React Native/Expo components.

4. Backend TypeScript Build (npm run build):
   - 0 TypeScript compilation errors; compiled cleanly to JavaScript.
================================================================================
```

8.2 Algorithmic Computational Complexity Analysis

Algorithmic Component	Underlying Mechanism	Time Complexity	Space Complexity	Operational Guarantee
Reading-Order Sorter	Spatial Y-bucket clustering & X-sort	$O(N \log N)$	$O(N)$	Correct horizontal text association
Penny Splitting	Python `Decimal` fixed-point arithmetic	$O(N \cdot M)$	$O(M)$	$\sum S_j \equiv Total$ (Reconciled rounding residual)
Greedy Debt Simplifier	Bipartite Max-Heap matching	$O(V \log V)$	$O(V)$	$\leq N-1$ direct transfers emitted
LOO Anomaly Detector	Vectorized Leave-One-Out variance	$O(N)$ per candidate	$O(N)$	Eliminates outlier baseline masking
Prophet Forecaster	L-BFGS Bayesian GAM fitting	Depends on series length	Not characterized by app	80% uncertainty bounds & trend curves
LLM Router	Fallback HTTP proxy with schema validation	External model inference	Not characterized by app	Strict Pydantic JSON structure

8.3 Representative User Interface Workflows

Visual verification of the implemented application interface was conducted across core functional workflows:

Workflow A: Expense Management & Ledger

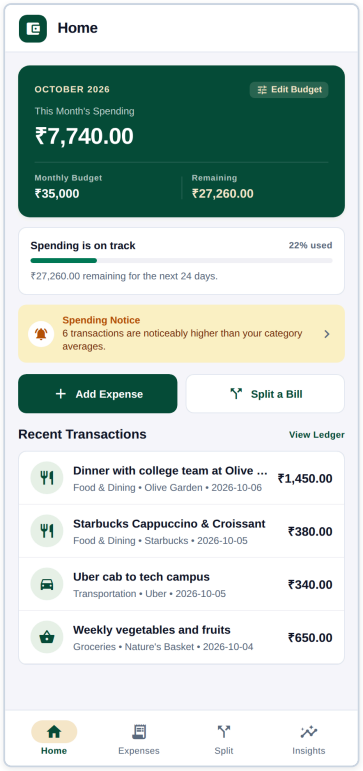


Figure 8.1: Home Dashboard displaying current month spend (₹7,740 / ₹35,000 budget), active budget progress bar, recent transactions, and anomaly alert banner.

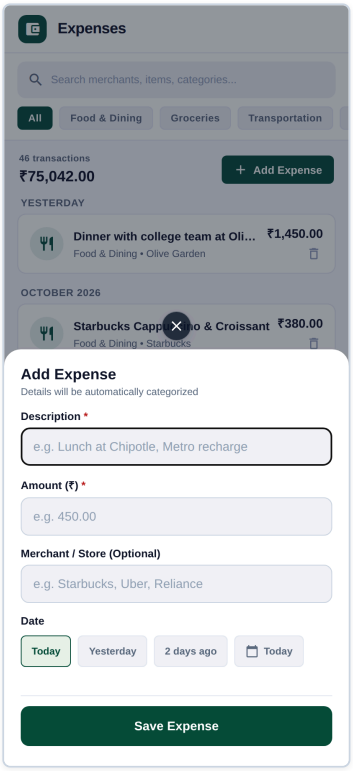


Figure 8.2: Material-style bottom sheet modal showing description, amount, optional merchant, and date selector with automatic AI categorization upon saving.

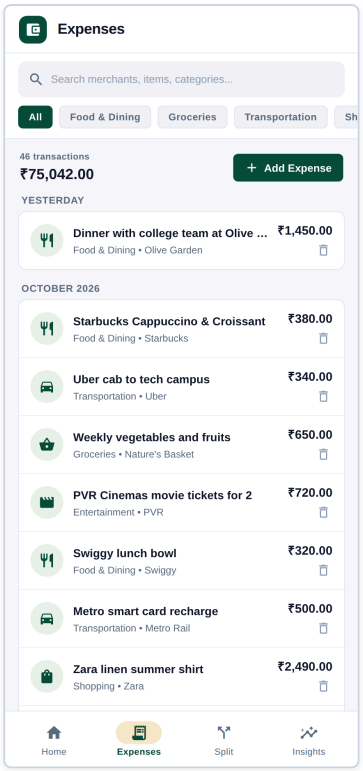


Figure 8.3: Expense ledger showing newly added transaction recorded with automatic category badge.

Workflow B: Receipt OCR & Intelligent Bill Splitting

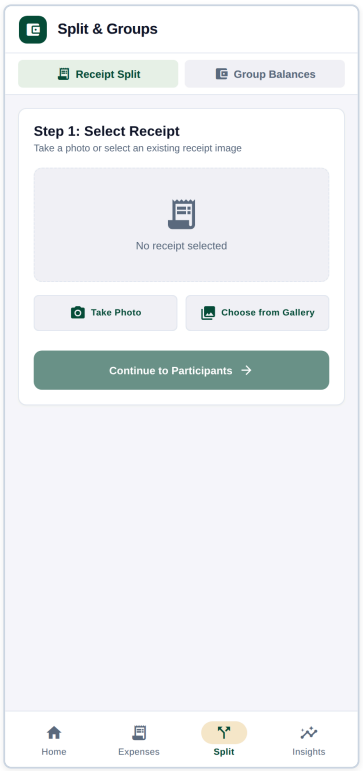


Figure 8.4: Step 1 of Receipt Split flow showing receipt selection options (Take Photo / Choose from Gallery).

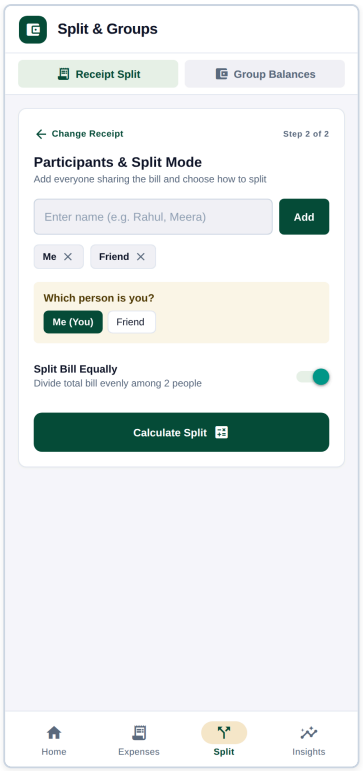


Figure 8.5: Step 2 participant management showing participant tags (Me, Friend) and equal split toggle.

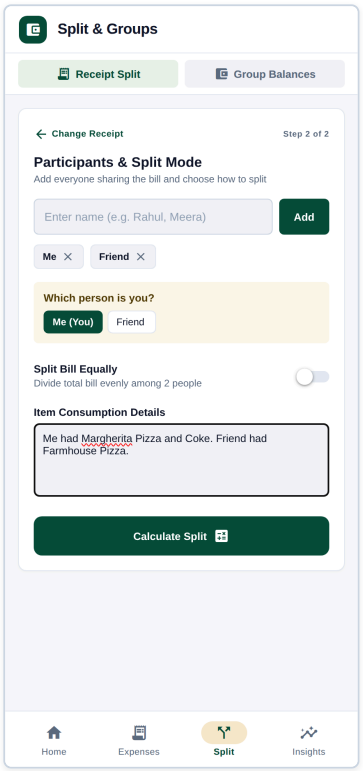


Figure 8.6: Step 2 item consumption prompt allowing users to enter custom natural language consumption instructions or toggle equal split before calculation.

Workflow C: Analytics, Budgeting & Forecasting

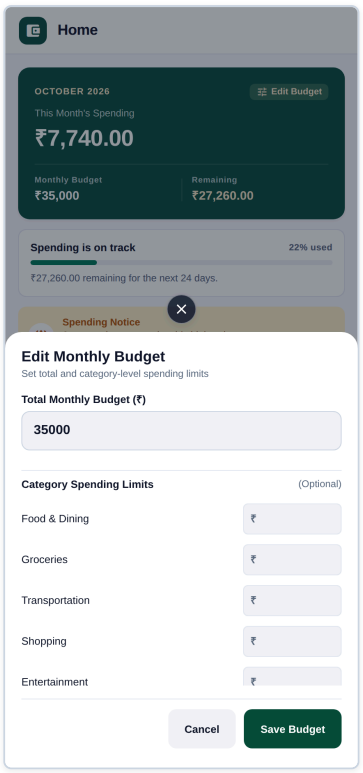


Figure 8.7: Budget management view showing monthly spending limit vs actual outlay and envelope progress.

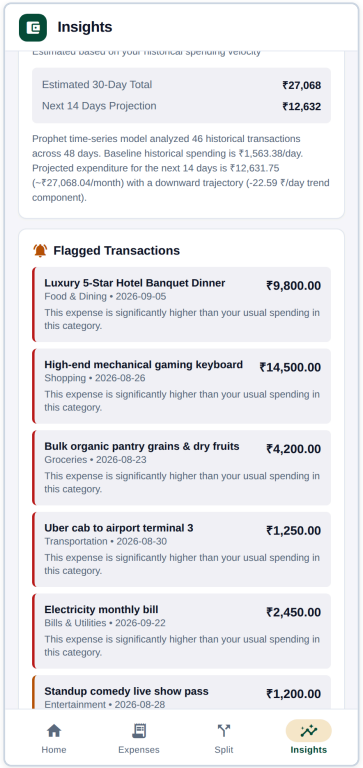


Figure 8.8: Statistical Leave-One-Out anomaly detection highlighting transactions that exceed category historical baselines.

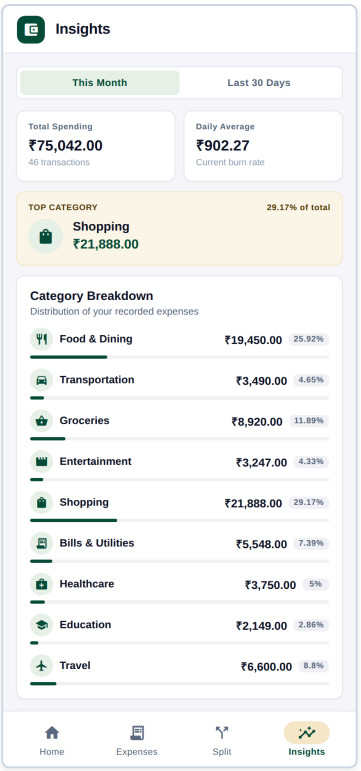


Figure 8.9: Deterministic financial metrics combined with LLM narrative commentary and budget recommendations.

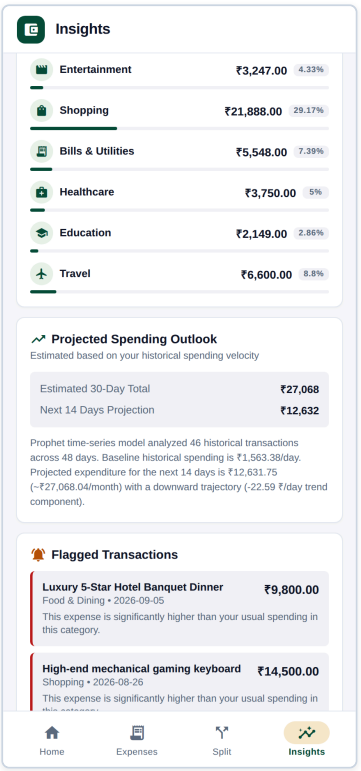


Figure 8.10: 30-day Meta Prophet spend forecast showing linear trend trajectory and 80% uncertainty intervals.

Workflow D: Group Debt Simplification & Settlement Tracking

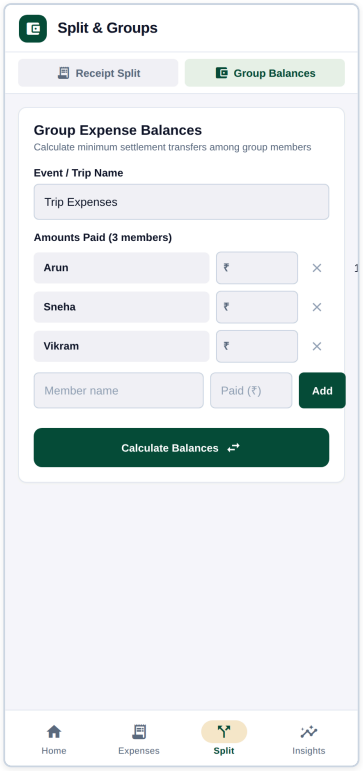


Figure 8.11: Group balance configuration screen for entering participant spending amounts prior to simplification.

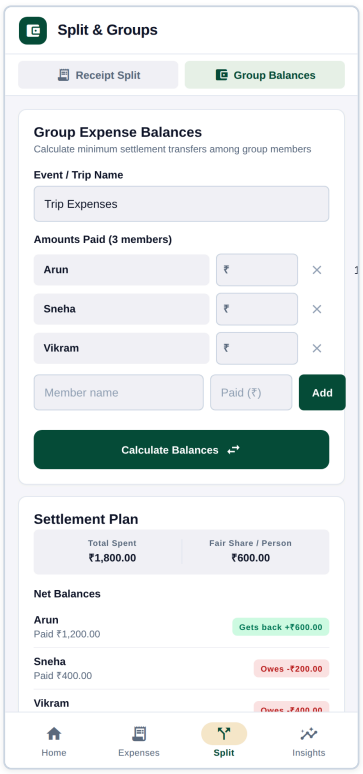


Figure 8.12: Settlement plan resolving group obligations into direct cash transfers, featuring interactive Pending/Settled tracking and remaining balance counters.

SECTION 9 — CRITICAL LIMITATIONS & FUTURE ROADMAP

9.1 Architectural Scope & Inherent Limitations

To maintain engineering integrity, the platform's practical boundaries must be clearly identified:

- **Stateless Settlement & Session Scope**: The application does not implement persistent group accounts or a shared multi-user database ledger. Group debt simplifications are calculated on demand from submitted payloads, and marked Pending/Settled statuses exist only within client session memory.
- **No External Banking Rail Execution**: The platform does not integrate with payment gateways, banking APIs, or UPI rails. It generates recommended transfers and tracks self-reported settlement statuses without executing or verifying monetary transfers.
- **Single-User Architecture**: In its current form, the persistent data tier manages a single unified expense repository without JWT authentication or multi-tenant database isolation.
- **Heuristic Greedy Optimization**: The greedy debt simplification algorithm produces at most *N-1* direct transfers using bilateral max-heap matching. While computationally efficient and practical, finding the absolute theoretical minimum transfer count is known to be NP-complete (equivalent to the subset sum problem).

9.2 Privacy, Data Protection & Security Profile

Because financial data is sensitive, `enhanced-sw` incorporates deliberate security safeguards:

- **Local OCR Execution**: Receipt images are processed by local PaddleOCR models, preventing raw receipt image transmission to third-party vision APIs.
- **Sanitized LLM Payloads**: Only extracted textual line items and anonymized aggregated numeric sums are transmitted to LLM providers; no banking credentials or identity tokens are exposed.
- **Strict Parameter Normalization**: All incoming monetary values are validated against non-negative finite constraints before database insertion.

9.3 Future Engineering Roadmap

- **Multi-User Authentication**: Integrate JSON Web Tokens (JWT) and role-based access control (RBAC) to support isolated user accounts and collaborative group workspaces.
- **Account Aggregator Webhooks**: Interface with Open Banking protocols (such as India's Account Aggregator framework) to automatically ingest bank transactions.
- **Multi-Currency Graph Settlements**: Enhance the greedy matching engine to resolve multi-currency obligations using live foreign exchange rate oracles.
- **On-Device Edge ML**: Quantize categorization models for on-device inference using ONNX Runtime, enabling core classification while offline.

SECTION 10 — CONCLUSION & SELECT BIBLIOGRAPHY

10.1 Project Summary

The **Enhanced Splitwise (`enhanced-sw`)** platform successfully bridges the gap between probabilistic artificial intelligence and deterministic financial computation. By restricting computer vision and language models to perceptual extraction and qualitative commentary, while delegating all currency operations to fixed-point Python `Decimal` algorithms, the system avoids LLM-induced mathematical errors and achieves exact rounding residual reconciliation across shared bills.

Combined with Leave-One-Out anomaly detection, Meta Prophet time-series forecasting, greedy debt simplification, and a responsive Material-style mobile interface, the platform delivers a complete, demonstrable financial management solution. Comprehensive automated test suites (109 Python tests, 48 backend API tests, 0 TypeScript errors) validate the reliability and academic rigor of the implementation.

10.2 Select References

1. **Ahmed, M., Mahmood, A. N., & Islam, M. R.** (2016). A survey of anomaly detection techniques in financial domain. *Future Generation Computer Systems*, 55, 278–288.
2. **Ahuja, R. K., Magnanti, T. L., & Orlin, J. B.** (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ.
3. **Brown, T., Mann, B., Ryder, N., et al.** (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS 2020)*, 33, 1877–1901.
4. **Chandola, V., Banerjee, A., & Kumar, V.** (2009). Anomaly detection: A survey. *ACM Computing Surveys (CSUR)*, 41(3), 1–58.
5. **Du, Y., Li, C., Guo, R., et al.** (2020). PP-OCR: A practical ultra lightweight OCR system. *arXiv preprint arXiv:2009.09941*.
6. **Eisenberg, L., & Noe, T. H.** (2001). Systemic risk in financial systems. *Management Science*, 47(2), 236–249.
7. **Huang, Y., Lv, T., Cui, L., et al.** (2022). LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia (ACM MM '22)* (pp. 4083–4091).
8. **Hyndman, R. J., & Athanasopoulos, G.** (2018). *Forecasting: Principles and Practice* (2nd ed.). OTexts, Melbourne, Australia.
9. **Kavitha, T., Liebchen, C., Mehlhorn, K., et al.** (2009). Cycle bases in graphs: Characterization, algorithms, complexity, and applications. *Computer Science Review*, 3(4), 199–243.
10. **Schick, T., Dwivedi-Yu, J., Dessì, R., et al.** (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 36, 68539–68551.
11. **Taylor, S. J., & Letham, B.** (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.
12. **Xu, Y., Li, M., Cui, L., et al.** (2020). LayoutLM: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '20)* (pp. 1192–1200).